Persistencia de Datos ORM

Elaborado por Jennifer Fajardo



@SENAComunica

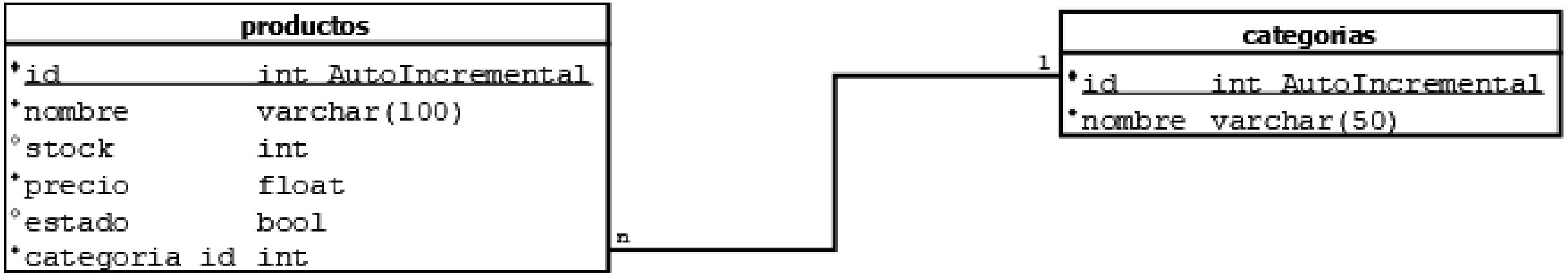
www.sena.edu.co

GC-F-004v.08

Trabajando con tablas relacionadas



Trabajando con tablas relacionadas



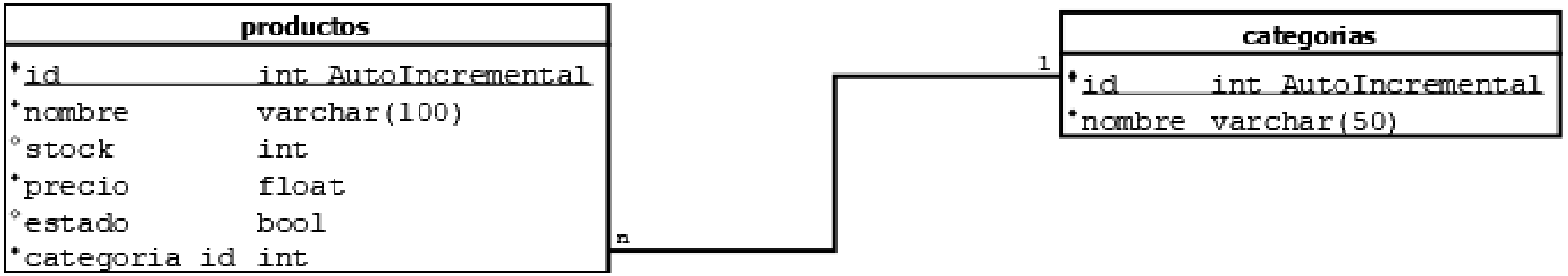
Reto 1 CRUD en 45 minutos

Empecemos creando la CRUD de Categorías y Registra al menos 5 categorías

Trabajando con tablas relacionadas

Cuando trabajamos con relaciones en un ORM como SQLAlchemy, debemos configurar **dos cosas diferentes**:

- **La Restricción Física (Foreign Key):** Le dice a MySQL que la columna **categoria_id** en la tabla de productos debe apuntar a un **id** real en la tabla de categorías.
- **La Relación Virtual (Relationship):** Es la magia del ORM. Le permite a Python hacer cosas como producto.categoria.nombre directamente, sin escribir un JOIN de SQL a mano.



Capa modelo



Modelo independiente Categoria

Primero deben crear el modelo independiente (**Categoría**)

```python
from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import relationship
from core.database import Base


class Category(Base):
    __tablename__ = "categorias"


    id = Column(Integer, primary_key=True, index=True)
    nombre = Column(String(50), unique=True, nullable=False)


    # Relación virtual: Una categoría tiene muchos productos
    # back_populates conecta esta relación con la que definiremos en Product
    products = relationship("Product", back_populates="category")
```

Modelo dependiente Producto

Primero deben crear el modelo independiente (**Categoría**)

```python
from sqlalchemy import Column, Integer, String, Float, ForeignKey
from sqlalchemy.orm import relationship
from core.database import Base


class Product(Base):
    __tablename__ = "productos"


    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    nombre = Column(String(100), unique=True, nullable=False)
    stock = Column(Integer, default=0)
    precio = Column(Float, nullable=False)


    # 1. La Llave Foránea Física en MySQL
    categoria_id = Column(Integer, ForeignKey("categorias.id", ondelete="CASCADE"), nullable=False)


    # 2. La Relación Virtual en Python (Para navegar entre objetos)
    category = relationship("Category", back_populates="products")
```

Capa schemas



Schema Producto

Se agrega el campo **categorías_id**

```
class ProductBase(BaseModel):
    nombre: str =Field(...,max_length=70,min_length=3, example="Lápiz")
    stock: int=Field(...,ge=0,examples=[10])
    precio:float=Field(...,ge=0,examples=[2030.5])
    categoria_id: int #Agregamos el campo de categoría para relacionar el producto con su categoría (FK)
```

Capa service



Servicio Producto

Vamos a requerir un método que llamaremos auxiliar para validar que cuando se cree o se actualice un producto, la categoría ingresada exista

```
from http.client import HTTPException, status

from models.category import Category


def _validar_categoria(self, categoria_id: int):
        """Método interno auxiliar para verificar si una categoría existe en la BD."""
        category_exists = self.db.query(Category).filter(Category.id == categoria_id).first()
        if not category_exists:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=f"Operación inválida: La categoría con ID {categoria_id} no existe."
            )
```

Servicio Producto

Llamamos el método auxiliar dentro del servicio para crear

```
def create(self, product_in: ProductCreate):
        """Crea un nuevo producto a partir del esquema de entrada y lo persiste."""
        # 1. VALIDACIÓN: Asegurar que la categoría asignada al nuevo producto exista
        self._validar_categoria(product_in.categoria_id)


        # 2. PERSISTENCIA: Si la categoría existe, procedemos a guardar el producto
        db_product = Product(**product_in.model_dump())


        self.db.add(db_product) # Agrega el nuevo producto a la sesion de trabajo
        self.db.commit()#  Guarda los cambios en la base de datos
        self.db.refresh(db_product) # Refresca el objeto con los datos actualizados (como el ID generado)
        return db_product
```

Servicio Producto

Llamamos el método auxiliar dentro del servicio para crear

```python
def update(self, product_id: int, product_in: ProductUpdate):
        """Actualiza un producto existente con los campos enviados."""
        db_product = self.get_by_id(product_id)
        if db_product:
            # Solo actualiza los campos que fueron enviados (exclude_unset=True)
            update_data = product_in.model_dump(exclude_unset=True)


            # === VALIDACIÓN DE LA LLAVE FORÁNEA ===
            # Si el usuario intenta modificar la categoría del producto, verificamos que sea válida
            if "categoria_id" in update_data:
                self._validar_categoria(update_data["categorias_id"])
            # ======================================
            for key, value in update_data.items():
                setattr(db_product, key, value)
            self.db.commit() # Guarda los cambios en la base de datos
            self.db.refresh(db_product) # Refresca el objeto con los datos actualizados
        return db_product
```

Actividades

Generar la CRUD de Categorías y registrar al menos 5 categorías
Completar la CRUD de Producto garantizando que se integre la llave foránea con categoría, registrar 10 productos que incluyan categorías válidas.
Generar las CRUD correspondientes a 5 módulos de su proyecto formativo integrando:

- ❑ Operaciones básicas conforme a los requisitos funcionales, se revisará el cumplimiento de reglas de negocio (validaciones campos, duplicados, reglas especiales)
- ❑ Próxima sesión revisión por grupo de funcionalidad

¿Dudas o Preguntas?



G R A C I A S

Línea de atención al ciudadano: 01 8000 910270
Línea de atención al empresario: 01 8000 910682

@SENAComunica

www.sena.edu.co